

SatQuery — System Architecture & Tech Stack

An AI-assisted geospatial console that lets users query satellite imagery and land-cover change over time in plain language, with map highlighting, a year timeline, region summaries, and proactive anomaly alerts.

1. Overview

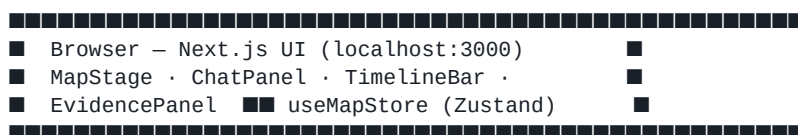
SatQuery is a two-tier web application: a React/Next.js "mission console" frontend and a Python/FastAPI analysis backend. Users draw a region or ask a question in plain language; the backend classifies satellite rasters into water / vegetation / built-up, compares years, and returns answers as both natural-language text (via Gemini) and map overlays (imagery, classification, highlights). It is a demo built around synthetic GeoTIFF rasters but uses real geospatial primitives so real imagery could be dropped in.

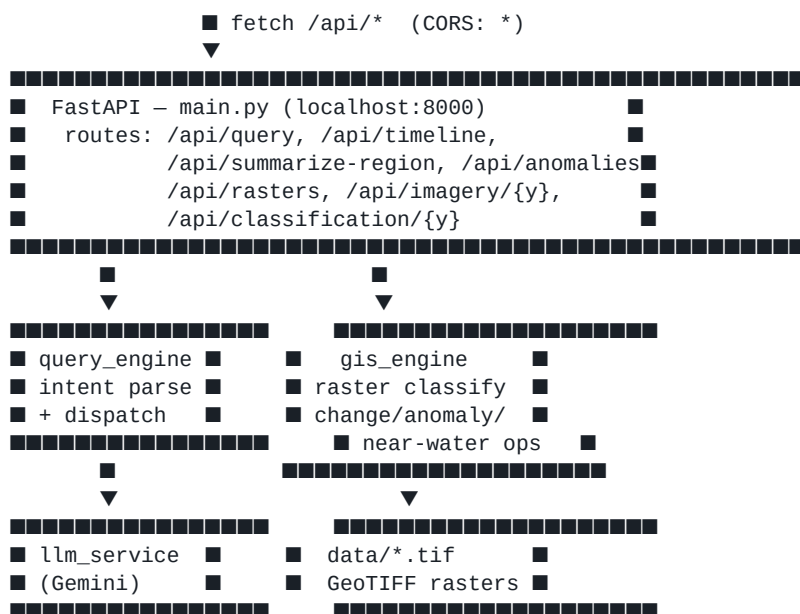
2. Tech Stack

Frontend framework	Next.js 16 (App Router, React 19, TypeScript)	UI shell, routing, rendering
Frontend styling	Tailwind CSS 4 + design tokens	"mission console" dark theme
Map	Leaflet 1.9 + react-leaflet 5	pan/zoom, image overlays, polygon drawing
State	Zustand	shared useMapStore across map/chat/timeline/evidence
Motion / icons	framer-motion, lucide-react	panel transitions, iconography
Backend framework	FastAPI + Uvicorn	REST API
Geospatial	rasterio, shapely, pyproj, NumPy	raster math, geometry, reprojection
Image rendering	Pillow	true-color + classification PNG overlays
AI	Google Gemini (google-genai)	plain-language narration of computed stats
Data	local GeoTIFF rasters (data/sentinel_*.tif)	50x50 synthetic demo imagery

3. System Architecture

Two independent processes. The Next.js server (port 3000) serves the UI; the FastAPI server (port 8000) exposes the analysis API. The browser calls the API directly over CORS.





Request flow: the frontend sends {geometry, query, start_year, end_year}. query_engine parses the intent (land-cover / change / near-water construction), runs the matching gis_engine operation, optionally narrates via Gemini, and returns {reply, intent, stats, highlights} plus map overlay URLs.

4. Backend Components

main.py	FastAPI app, routes, Pydantic schemas, error mapping
gis_engine.py	classification, true-color + class-mask rendering, temporal change, anomaly scan, near-water mask, GeoJSON vectorization
query_engine.py	keyword-based intent parsing and operation dispatch
llm_service.py	Gemini client wrapper; converts GIS stats into plain-language answers
scripts/generate_demo_data.py	regenerates the synthetic rasters (river + growing built-up)

5. API Reference

GET	/api/rasters	available years + extents
GET	/api/imagery/{year}	true-color PNG (full extent)
GET	/api/classification/{year}	water/veg/built-up overlay PNG

POST	/api/query	intent dispatch; returns reply + highlights + stats
POST	/api/timeline	change narrative + diff between years
POST	/api/summarize-region	region stats + narrative
GET	/api/anomalies	computed anomaly alerts

6. How Analysis Works

- **Classification** — per pixel, $NDVI = (NIR - RED) / (NIR + RED)$ and $NDWI = (GREEN - NIR) / (GREEN + NIR)$. Water: $NDWI > 0$. Vegetation: $NDVI > 0.3$. Built-up: the remaining valid land. Class masks are vectorized to GeoJSON for map highlights.
- **Change detection** — class percentages for two years are compared; net change per class is reported and narrated.
- **Near-water construction** — the end-year water mask is buffered, then intersected with pixels that became built-up since the start year.
- **Anomalies** — a 5x5 grid is scanned; cells with the largest built-up/vegetation change are flagged with real coordinates.

7. Frontend Structure

app/workspace/page.tsx	workspace route
components/system/	design primitives (GlassPanel, Button, StatBar, StatChip, Cursor, ...)
components/workspace/	MapStage, LeafletMap, ChatPanel, TimelineBar, EvidencePanel, Header
lib/api.ts	typed backend client + imagery URL builders
lib/store.ts	Zustand useMapStore (year, geometry, highlights, evidence, anomalies)

8. The Five Features

Ask what's in an image	classification overlay + highlighted polygons + text reply
Change over time	year slider swaps imagery; Gemini narrates computed change

Draw a region	polygon drawing triggers an instant region summary
Plain-language spatial queries	intent parser dispatches to the right operation
Proactive anomalies	grid-based change scan flags cells; jump-to before/after

9. Running It

```
# Terminal 1 — backend
cd ~/Downloads/satquery
.venv/bin/uvicorn main:app --port 8000
```

```
# Terminal 2 — frontend
cd ~/Downloads/satquery\ 2
npm run dev # open http://localhost:3000/workspace
```

The frontend targets `http://localhost:8000` by default; override with `NEXT_PUBLIC_API_BASE`.

10. Known Limitations

- Demo imagery is synthetic 50x50 rasters (a river + growing built-up), not real satellite data.
- The "AI" narrates computed statistics; there is no true object detection or vision model on the imagery.
- No authentication, authorization, or rate limiting (CORS is open for development).
- The model name in `llm_service.py` is a demo placeholder; swap for a valid Gemini model id as needed.